State Design Pattern

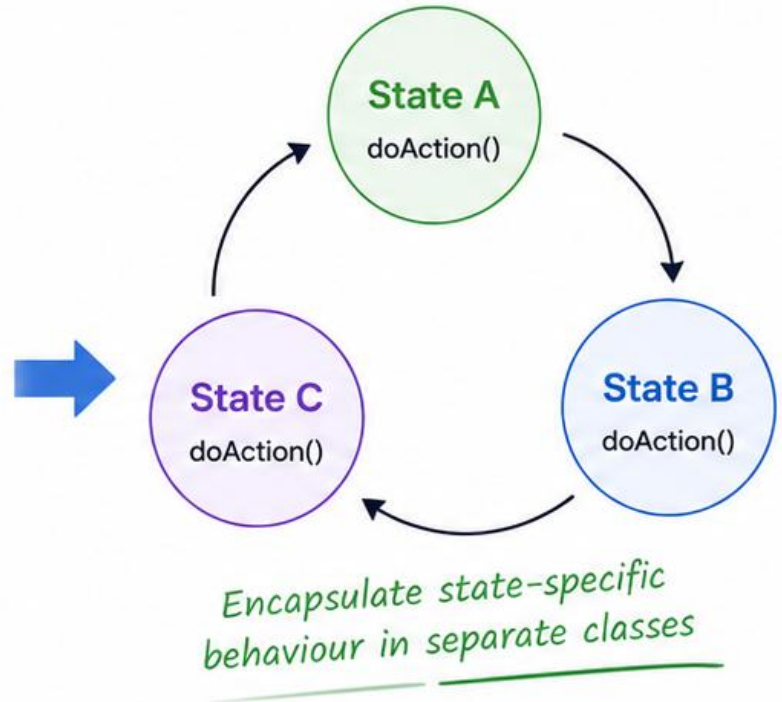
From if/else to State-Based Behaviour

Behavioural Design Pattern

> How can an object change its behaviour when its state changes?

Traditional Approach (if / else)

```
if (state == A)
    ...
else if (state == B)
    ...
else if (state == C)
    ...
else
    ...
```



ROADMAP: OUR JOURNEY SO FAR

CREATIONAL
How objects are created

Singleton → Factory → Abstract Factory (Upcoming)

STRUCTURAL
How objects are composed

Facade → Decorator → Proxy → Adapter

BEHAVIOURAL
How objects behave

Template → Chain of Responsibility → State — TODAY

OUR DESIGN THINKING FLOW

Think
Understand the problem

Model
Model states and transitions

Design
Design the state pattern solution

Defend
Evaluate and defend the design

Before You Come to Class

Your Pre-Class Preparation

Before the session:

1. Watch the assigned State Pattern video
2. Review the previous lecture notes
3. Read the short State Pattern extract
4. Attempt Exercise 01
5. Bring your state diagram to class

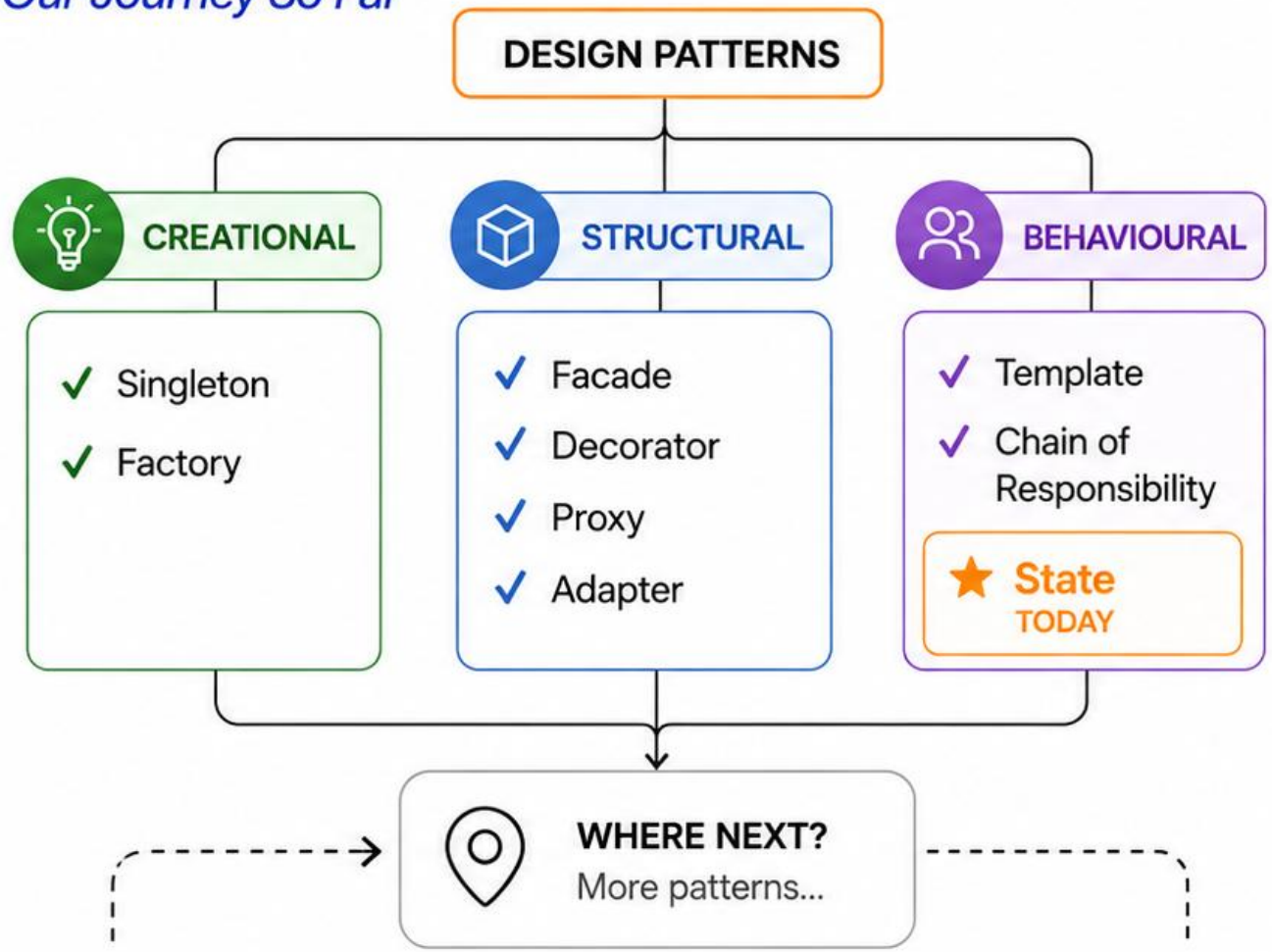
Don't try to memorise the pattern.
Instead, ask:

> **What behaviour changes when the object's state changes?**



Design Patterns:
Where Are We Now?

Our Journey So Far



Our Learning Journey

CREATIONAL	Singleton → Factory
STRUCTURAL	Facade → Decorator → Proxy → Adapter
BEHAVIOURAL	Template → Chain of Responsibility → State ★

Today

STATE DESIGN PATTERN

Our journey has moved from:

Creating objects → Structuring objects → Managing behaviour

Activate Prior Learning

Consider this code

```
1   if (status == NEW) {
2       ...
3   } else if (status == PAID) {
4       ...
5   } else if (status == SHIPPED) {
6       ...
7   } else if (status == DELIVERED) {
8       ...
9   } else if (status == CANCELLED) {
10      ...
    }
```

Question

What happens when we add:

RETURNED

or

REFUNDED

Think individually

What part of the code needs to change?

The Real Problem

The problem is NOT the `if`

The problem is that:

State-specific behaviour is concentrated in one place.

```
1   if (status == NEW) {
2       ...
3   } else if (status == PAID) {
4       ...
5   } else if (status == SHIPPED) {
6       ...
7   } else if (status == DELIVERED) {
8       ...
9   } else if (status == CANCELLED) {
10      ...
n   }
```

All state behaviour in one place

Ask yourself:

Who should own the behaviour of a particular state?

As states increase:

- More States
- More Conditions
- More Responsibilities
- Harder to Maintain
- Harder to Extend

Recognising a State-Based Problem

Is This Really a State?

A state is more than a value.

An object's state determines:

- What it **can do**
- How it **responds** to an action
- What state it can **move to next**

Example — TV Remote

The book's example models a TV with three states:

OFF → ON → MUTE

The same button can produce different results depending on the **current state**.

SAME BUTTON, DIFFERENT RESULT

Pressing the **POWER** button produces a different result depending on the **current state**.



OFF

- Press POWER → Turn TV ON
- Next state → ON

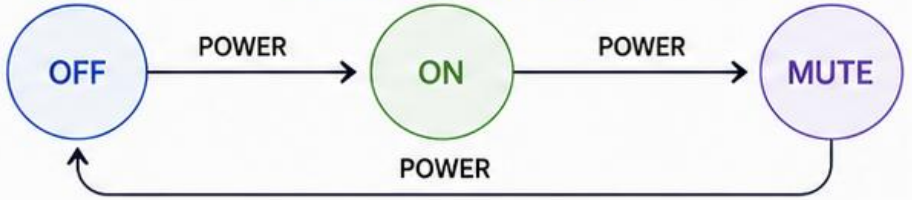
ON

- Press POWER → Mute the TV
- Next state → MUTE

MUTE

- Press POWER → Turn TV OFF
- Next state → OFF

State transition diagram

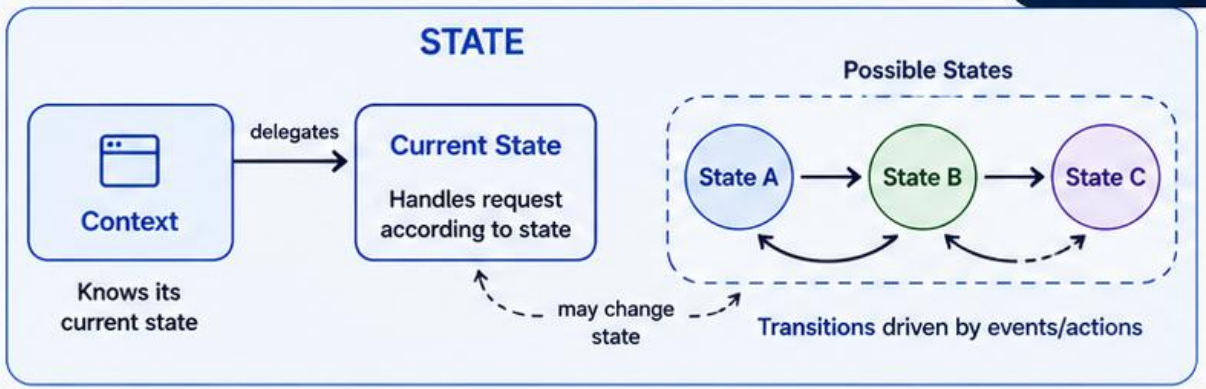


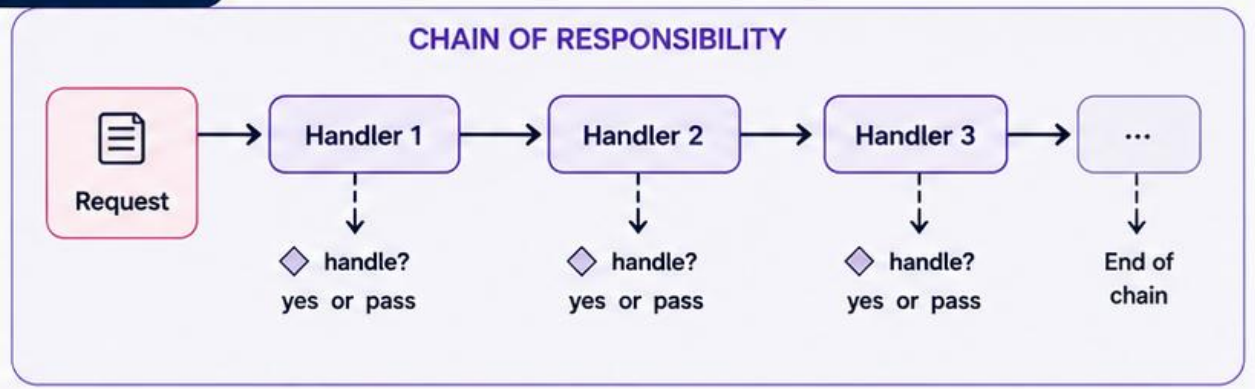
State vs Chain of Responsibility

Both delegate behaviour — but for different reasons

	STATE	CHAIN OF RESPONSIBILITY
Main question	How should I **behave** in this **state**?	Who should **handle** this **request**?
Flow	Behaviour is delegated to the **current state**	Request moves through a **sequence** of handlers
Decision	State decides: **respond** and/or **transition**	Handler decides: **handle** or **pass on**
Relationship	**Context** ↔ **Current State**	**Handler** → **next Handler**
Typical change	Add/change **states** or **transitions**	Add/remove/reorder **handlers**
Example	**NEW** → **PAID** → **SHIPPED** → **DELIVERED**	**L1 Support** → **L2 Support** → **L3 Support**
Key idea	Change behaviour by **changing state**	**Pass** the request along

The visual distinction





The one-line distinction: | State: "How should I behave right now?" | Chain of Responsibility: "Who will handle this request?"

Same Action ≠ Same Behaviour

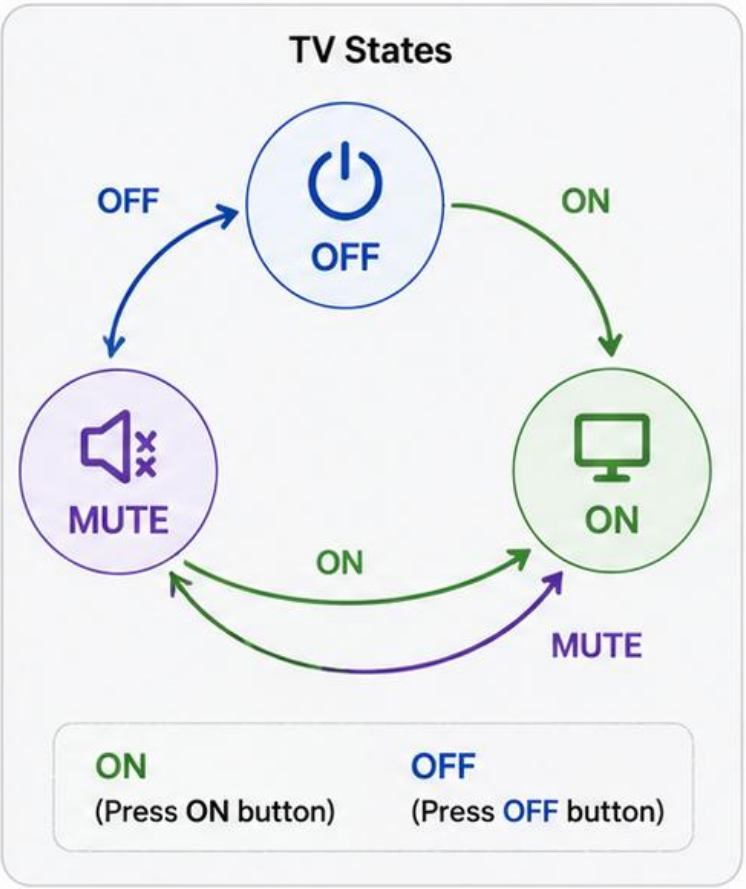
Press ON

Current State	Action	Result
OFF	ON	→ ON
ON	ON	No state change
MUTE	ON	→ ON

Press OFF

Current State	Action	Result
OFF	OFF	No state change
ON	OFF	→ OFF
MUTE	OFF	→ OFF





Key observation

The action is the same.

The behaviour depends on the current state.

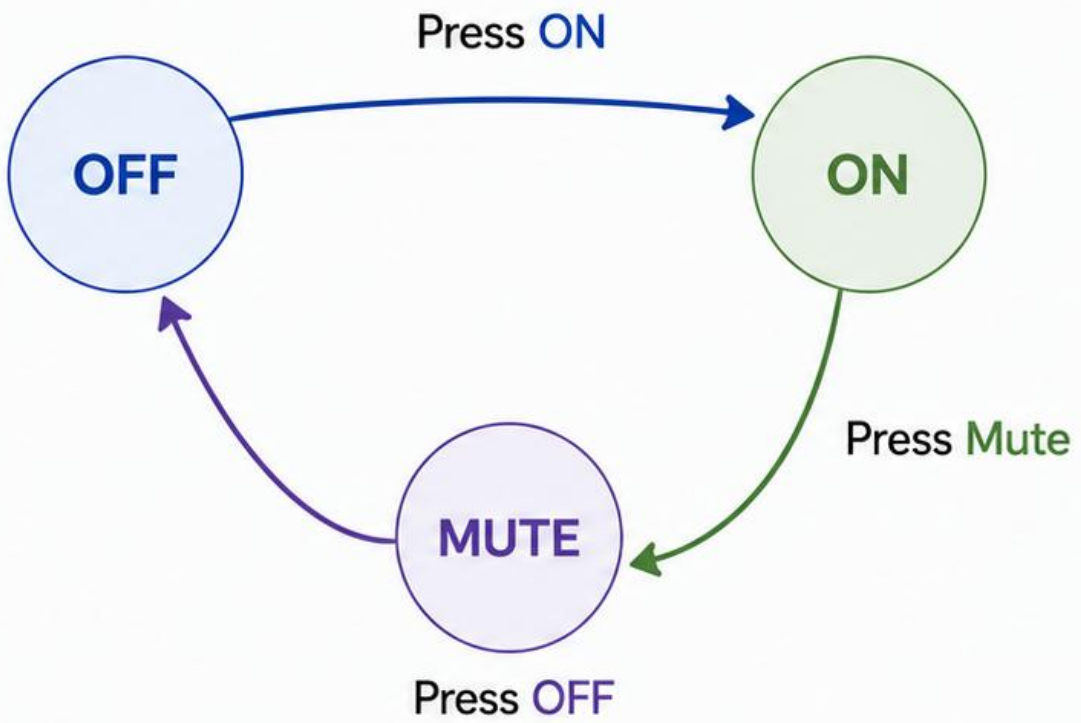
State Machine First

Before writing classes — draw the states.

For every transition ask:

State + Event → Behaviour → Next State

This is the foundation of the State Pattern.



Transitions

Current State	Event	Behaviour (What happens)	Next State
OFF	Press ON	Turn the TV ON	ON
ON	Press Mute	Mute the TV (sound OFF)	MUTE
MUTE	Press OFF	Turn the TV OFF	OFF

State Machine First

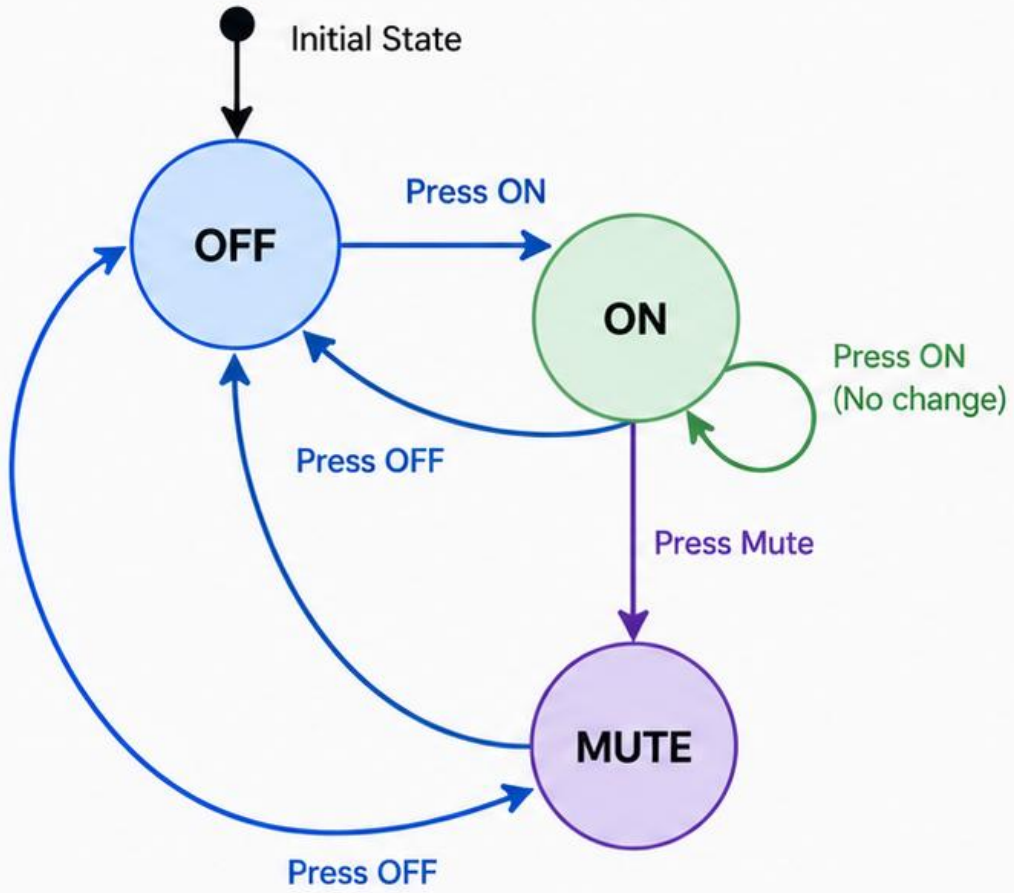
Before writing classes — draw the states.

Transition	State	Event	Behaviour	Next State
	OFF	Press ON	Turn TV ON	ON
	ON	Press ON	No state change	ON
	MUTE	Press ON	Turn TV ON	ON
	OFF	Press OFF	No state change	OFF
	ON	Press OFF	Turn TV OFF	OFF
	MUTE	Press OFF	Turn TV OFF	OFF

For every transition ask:

State + **Event** → **Behaviour** + **Next State**

This is the foundation of the State Pattern.

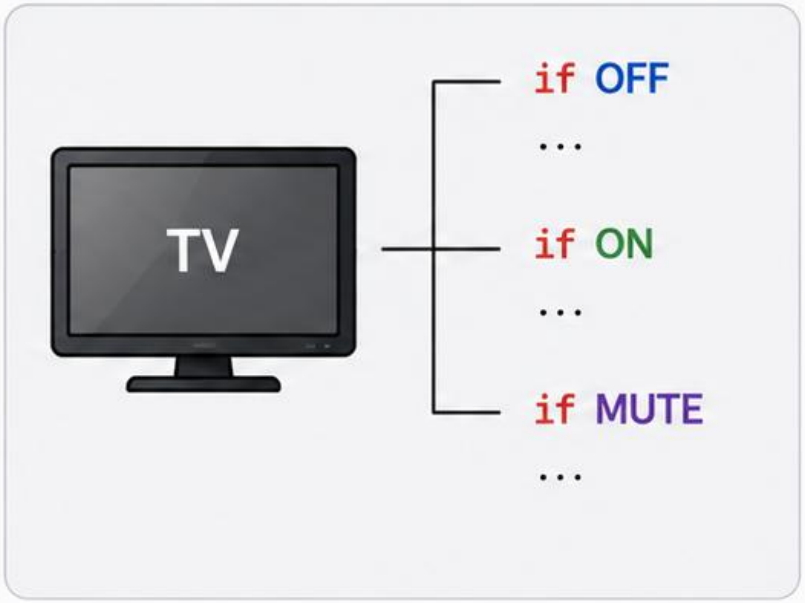


- **OFF** – TV is off
- **ON** – TV is on
- **MUTE** – TV is muted

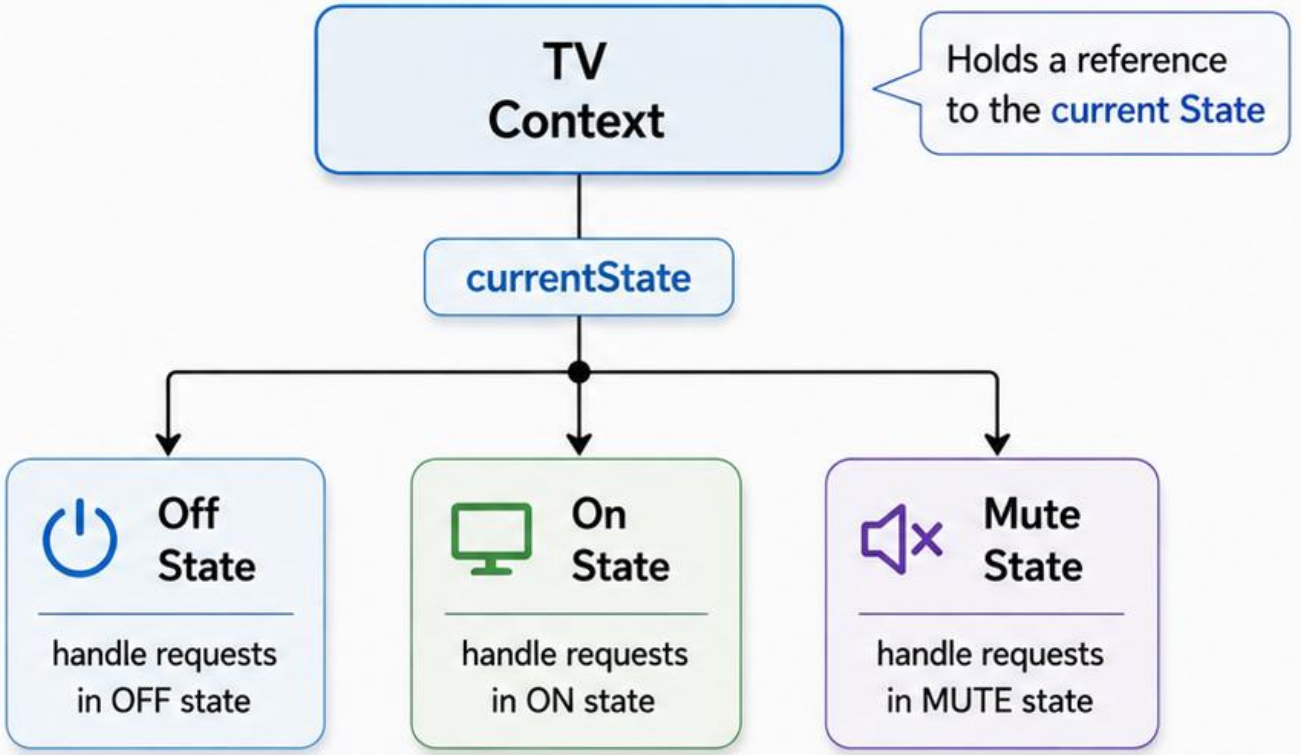
From State Machine to Objects

The Key Design Move

Instead of this:



We move behaviour into:

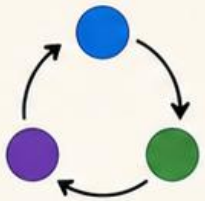




The Context asks:

"Current State, please handle this request."

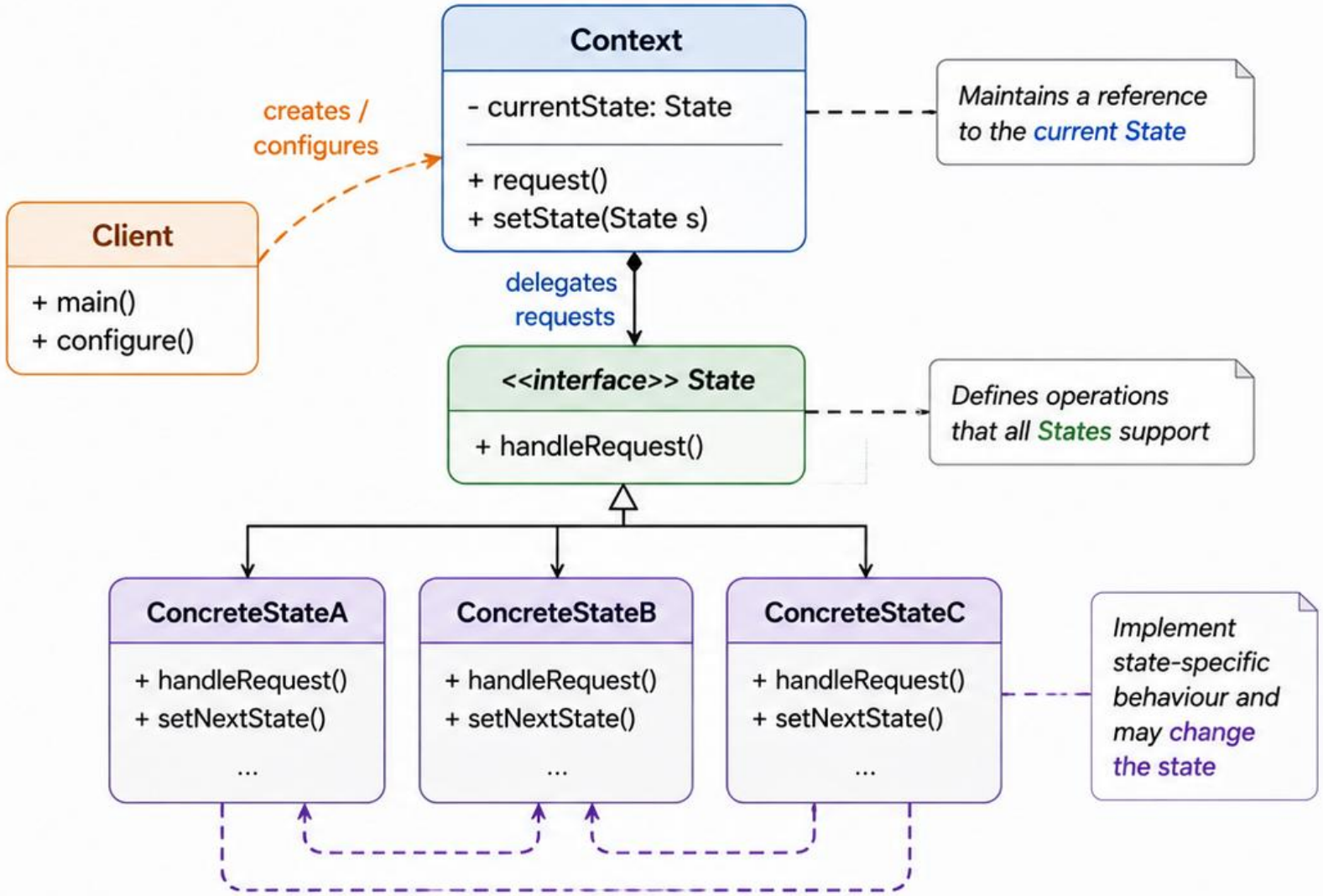
The current State object knows how to respond and may decide the next State.



The State Pattern Structure

Four important elements

1. **Context**
   - Maintains the current state
   - Delegates requests
2. **State**
   - Defines common operations
3. **Concrete States**
   - Implement state-specific behaviour
   - Can trigger state transitions
4. **Client**
   - Creates/configures the Context



The previous lecture already introduced this *Context–State structure*.

The State Pattern Structure

Object relationships and responsibilities

1 Context

- Maintains a reference to a State (currentState)
- Delegates requests to the current State

2 State (Interface)

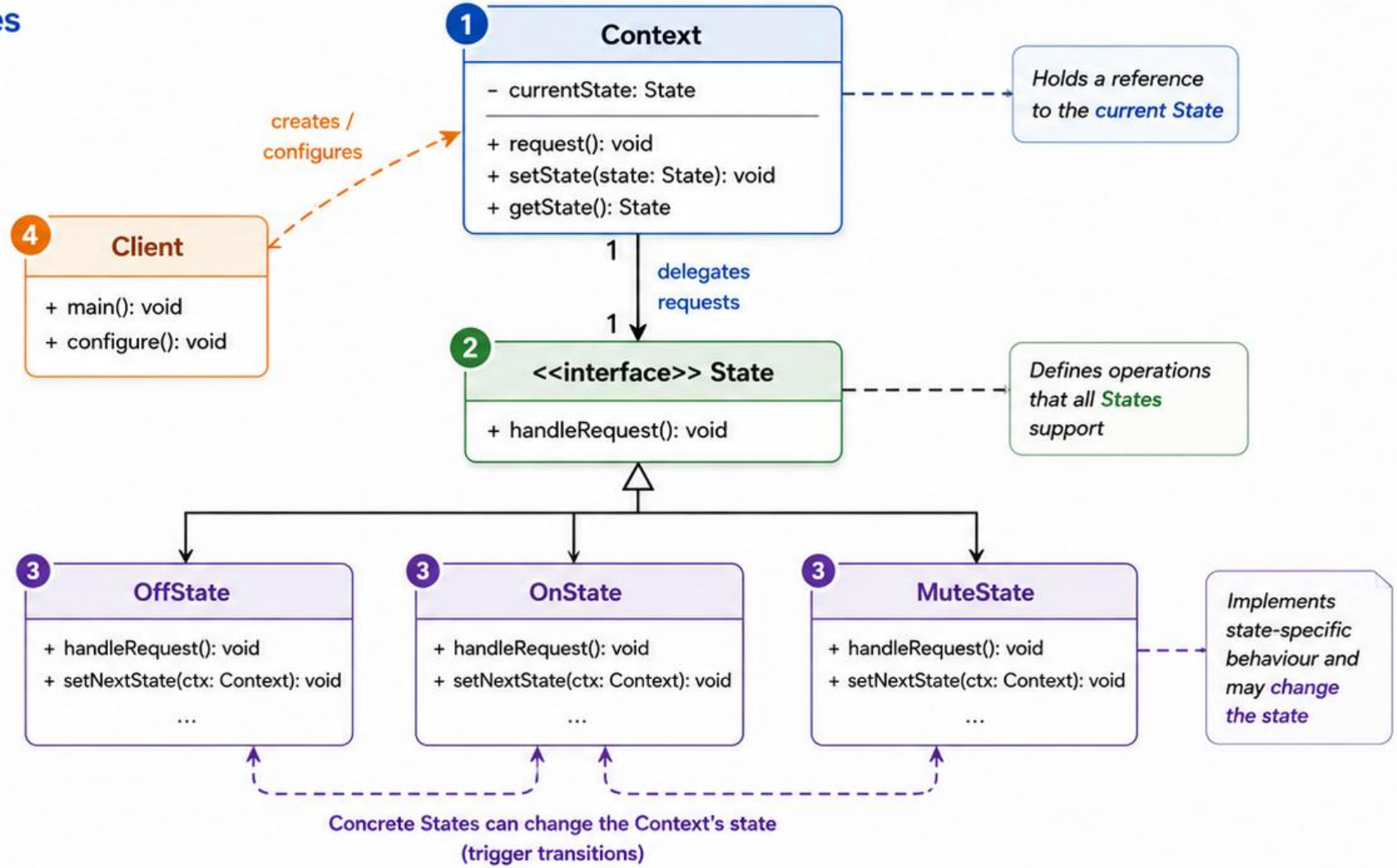
- Declares operations that all Concrete States support
- Provides a common type for all Concrete States

3 Concrete States

- Implement state-specific behaviour
- Can trigger state transitions by setting a new State in the Context

4 Client

- Creates and configures the Context
- Initiates requests to the Context



Key idea: The Context does not ask *"Which state am I?"*

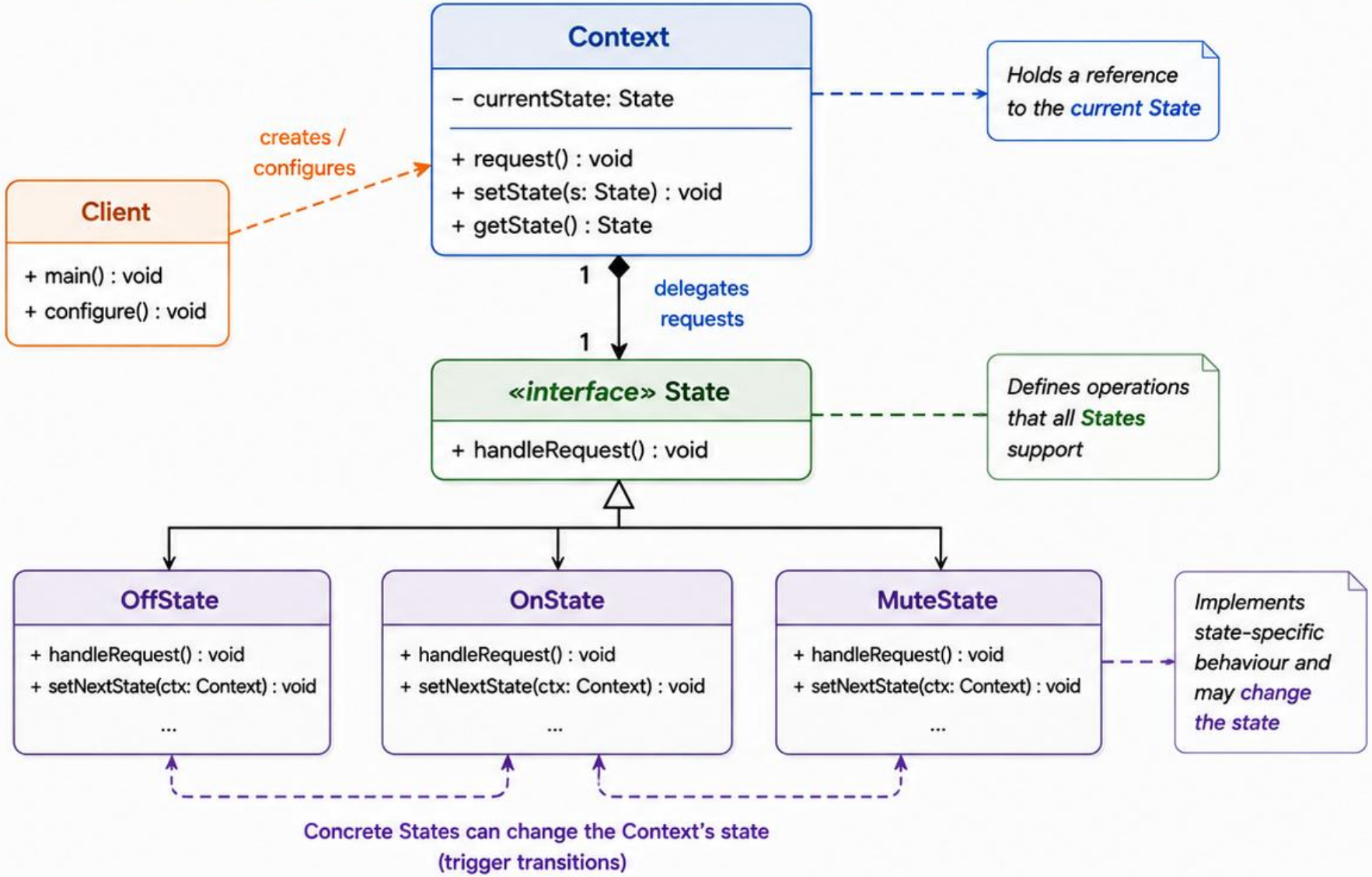
It asks the current State to handle the request.



The State Pattern Structure

Four important elements

1. **Context**
   - Maintains the current state
   - Delegates requests
2. **State**
   - Defines common operations
3. **Concrete States**
   - Implement state-specific behaviour
   - Can trigger state transitions
4. **Client**
   - Creates/configures the Context

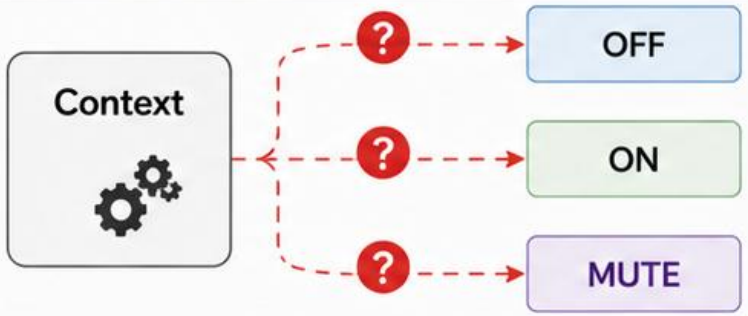


The previous lecture already introduced this *Context–State structure*.

Delegation Replaces Conditional Logic

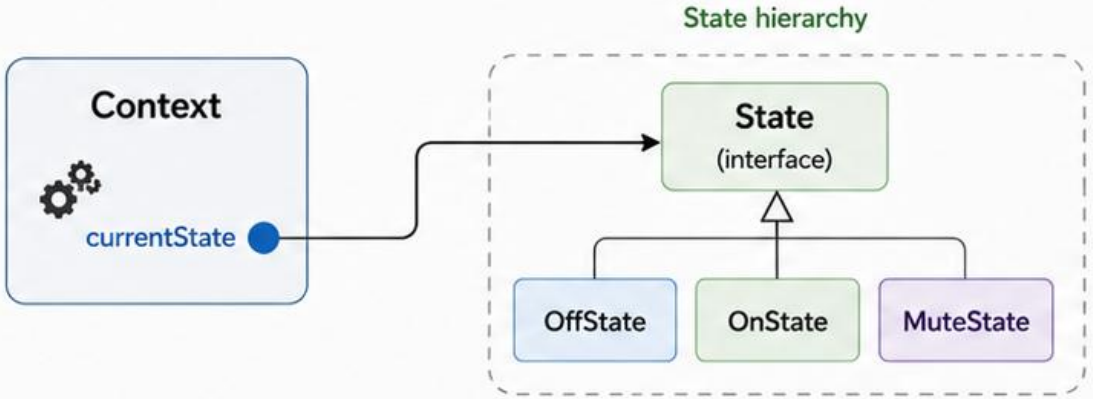
BEFORE The Context checks the state and decides what to do

```
public void pressButton() {
    if (currentState == OFF) {
        ...
    }
    else if (currentState == ON) {
        ...
    }
    else if (currentState == MUTE) {
        ...
    }
}
```



AFTER The Context delegates the request to the current state

```
public void pressButton() {
    currentState.pressButton(this);
}
```



The Context no longer asks:

"Which state am I?"

It asks:

"Current state, what should I do?"

Exercise 01 — Pre-Class Preparation

Exercise 01: Design the TV Remote

Individual Preparation

The TV can be in:

- OFF
- ON
- MUTE

The remote has:

- pressOnButton()
- pressOffButton()
- pressMuteButton()

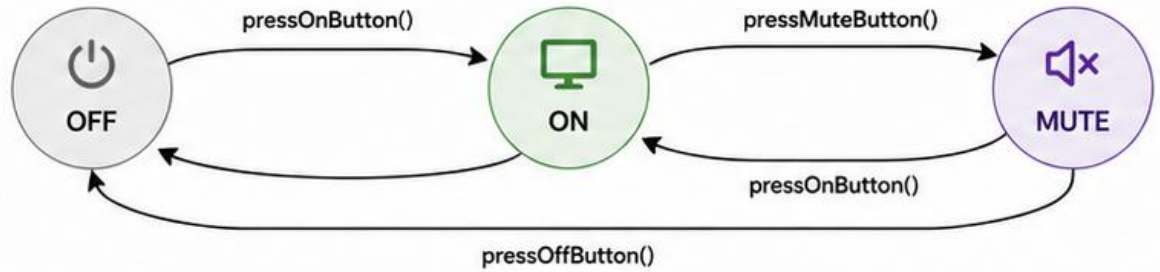
Your task

Design the State Pattern solution.

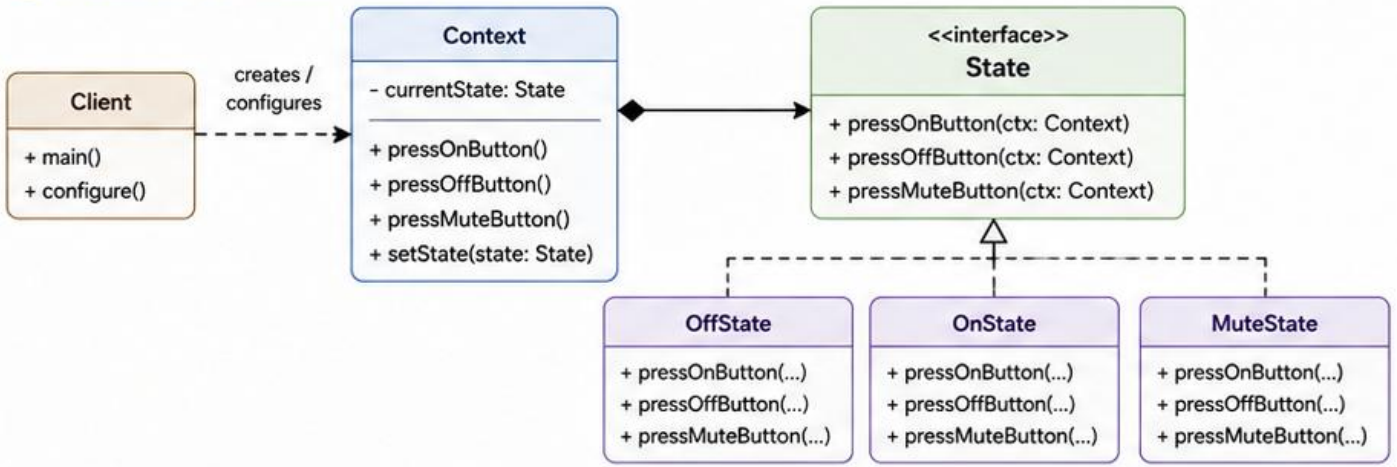
- Identify the states, events and valid transitions.
- Draw the state machine.
- Design the classes using the State Pattern structure.
- Show how state transitions occur.
- Write brief notes on each class responsibility.

Design the State Pattern Solution

1 State Machine (TV States and Transitions)



2 Class Diagram (State Pattern Structure)



Think, design and prepare your sketches. We will discuss your solutions in class.

Exercise 01: Step 1 — Build the State Table

Goal: Define the behaviour and next state for every action.

TV States

OFF	TV is turned off
ON	TV is turned on (sound on)
MUTE	TV is on but muted

Rule

If the action should have no effect, explicitly record:

No state change

This follows the TV-state example in the book extract.

Complete the table:

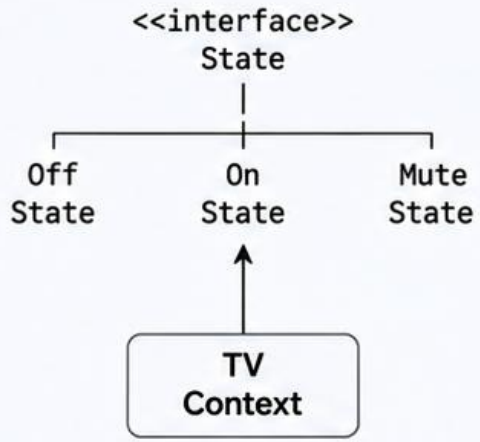
Current State	Action	Behaviour (What happens?)	Next State
OFF	ON	?	?
	OFF	?	?
	MUTE	?	?
ON	ON	?	?
	OFF	?	?
	MUTE	?	?
MUTE	ON	?	?
	OFF	?	?
	MUTE	?	?

Exercise 01: Step 2 — Draw the Class Design

Create the classes

Design the State Pattern structure for the TV remote.

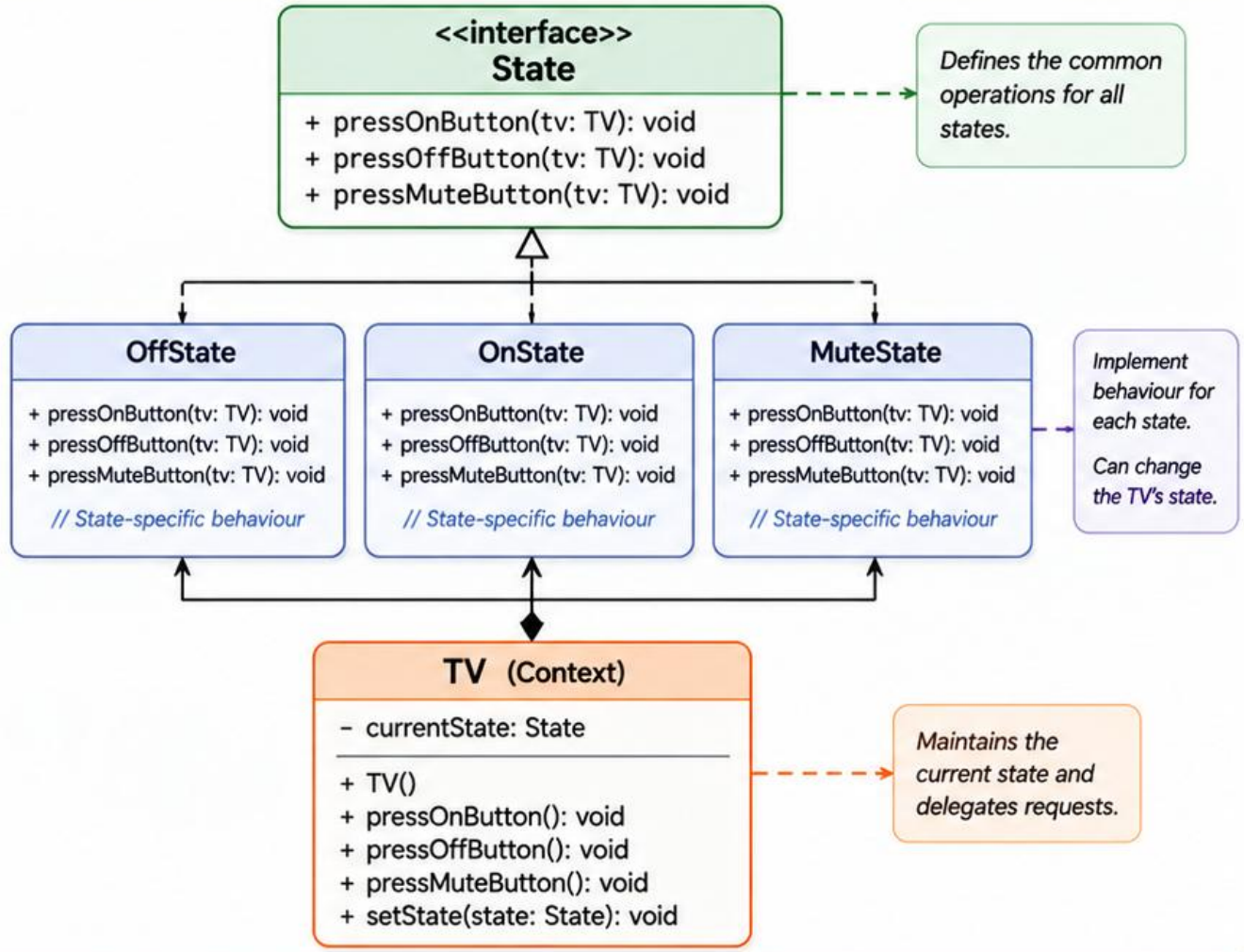
Target structure:



Guidance

- Use the State Pattern structure discussed in the lecture.
- Apply it to the TV example.

UML Class Diagram



Decide:

1. What methods belong in **State**?
2. What does **TV** store?
3. Who changes the **current state**?
4. Which class performs the **actual behaviour**?

Tip: Start from the **state table** you completed in the previous step. Use it to decide the behaviour of each **Concrete State**.

Exercise 01: Step 3 — Trace the Behaviour

Initial State



TV = OFF

Execute the Following Actions

ON

↓

MUTE

↓

ON

↓

OFF

↓

OFF

Trace the Behaviour

Step	Action (Button Pressed)	Who Receives the Request?	Which State Object Handles It?	Who Performs the Transition?	Resulting State
Initial	–	TV (Context)	OffState	OffState → OnState	OFF
1	ON	TV (Context)	OffState	OffState → OnState	ON
2	MUTE	TV (Context)	OnState	OnState → MuteState	MUTE
3	ON	TV (Context)	MuteState	MuteState → OnState	ON
4	OFF	TV (Context)	OnState	OnState → OffState	OFF
5	OFF	TV (Context)	OffState	No state change	OFF

Challenge

For each step, identify:

- Who receives the request?
- Which state object handles it?
- Who performs the transition?

Remember:

The Context delegates to the current State object to handle the request.

State Pattern – TV Remote Example (Java Implementation)

1) State Interface

```
public interface State {
   void pressOnButton(TV tv);
   void pressOffButton(TV tv);
   void pressMuteButton(TV tv);
}
```

Defines the operations that all concrete states must implement.

2) Concrete States

OffState.java

```
public class OffState implements State {

  @Override
  public void pressOnButton(TV tv) {
     System.out.println("TV turned ON");
     tv.setState(tv.getOnState());
  }

  @Override
  public void pressOffButton(TV tv) {
     System.out.println("TV is already OFF");
     // No state change
  }

  @Override
  public void pressMuteButton(TV tv) {
     System.out.println("Cannot mute while OFF");
     // No state change
  }
}
```

OnState.java

```
public class OnState implements State {

  @Override
  public void pressOnButton(TV tv) {
     System.out.println("TV is already ON");
     // No state change
  }

  @Override
  public void pressOffButton(TV tv) {
     System.out.println("TV turned OFF");
     tv.setState(tv.getOffState());
  }

  @Override
  public void pressMuteButton(TV tv) {
     System.out.println("TV is now MUTE");
     tv.setState(tv.getMuteState());
  }
)
```

MuteState.java

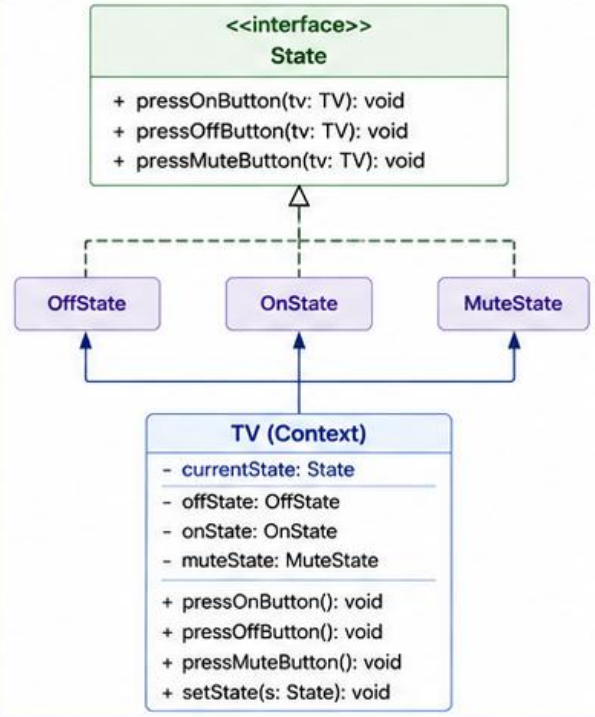
```
public class MuteState implements State {

  @Override
  public void pressOnButton(TV tv) {
     System.out.println("TV ON (unmuted)");
     tv.setState(tv.getOnState());
  }

  @Override
  public void pressOffButton(TV tv) {
     System.out.println("TV turned OFF");
     tv.setState(tv.getOffState());
  }

  @Override
  public void pressMuteButton(TV tv) {
     System.out.println("TV is already MUTE");
     // No state change
  }
}
```

Class Design (Structure)



3) Context Class (TV)

```
public class TV {
   private State currentState;
   private State offState;
   private State onState;
   private State muteState;

   public TV() {
       offState  = new OffState();
       onState   = new OnState();
       muteState = new MuteState();
       currentState = offState;   // Initial state
   }
   // Set current state
   public void setState(State state) { this.currentState = state; }

   // Getters for states (used by concrete states for transitions)
   public State getOffState()     { return offState;   }
   public State getOnState()      { return onState;    }
   public State getMuteState()    { return muteState;  }

   // Delegate button presses to the current state
   public void pressOnButton()  { currentState.pressOnButton(this); }
   public void pressOffButton() { currentState.pressOffButton(this); }
   public void pressMuteButton(){ currentState.pressMuteButton(this); }
}
```

4) Client / Test (Demo)

```
public class TVRemoteDemo {
   public static void main(String[] args) {
      TV tv = new TV();      // Starts in OFF state

      System.out.println("Initial State: OFF");
      System.out.println("------------------------");

      tv.pressOnButton();     // OFF  -> ON
      tv.pressMuteButton();   // ON   ->  MUTE
      tv.pressOnButton();     // MUTE -> ON
      tv.pressOffButton();    // ON   -> OFF
      tv.pressOffButton();    // OFF  -> OFF (no change)
  }
}
```

How It Works

1. TV (Context) holds the current state object.
2. When a button is pressed, TV delegates the request to the current state.
3. The state object performs the behaviour and may change the state by calling tv.setState(...).
4. The Context never asks "Which state am I?" It simply asks the current state to handle the request.

5) Sample Output

```
Initial State: OFF
------------------------
TV turned ON
TV is now MUTE
TV ON (unmuted)
TV turned OFF
TV is already OFF
```

Note:
This demonstrates how the State Pattern replaces conditional logic with delegation.

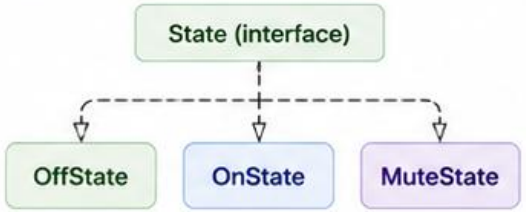
State Pattern – TV Remote Example (Java)

We will understand using 4 simple ideas: Interface → Classes → Constructor → Objects (with runtime flow)

1 INTERFACE (State)

Defines the operations that all states must support.

```java
public interface State {
    void pressOnButton(TV tv);
    void pressOffButton(TV tv);
    void pressMuteButton(TV tv);
}
```



All concrete states implement this interface.

2 CLASSES (Concrete States)

Each class implements State and provides its own behaviour.

```java
public class OffState implements State {
    // behaviour when TV is OFF
    ...
}
```

```java
public class OnState implements State {
    // behaviour when TV is ON
    ...
}
```

```java
public class MuteState implements State {
    // behaviour when TV is MUTE
    ...
}
```

Each class decides what to do for ON, OFF and MUTE actions and when to change to another state.

3 CONSTRUCTOR (TV)

Creates the state objects and sets the initial state.

```java
public class TV {
    private State currentState;
    private State offState;
    private State onState;
    private State muteState;

    public TV() {
        offState    = new OffState();
        onState     = new OnState();
        muteState   = new MuteState();
        currentState = offState;    // initial state
    }
  // button methods delegate to current state

  public void pressOnButton()   { currentState.pressOnButton(this); }
  public void pressOffButton()  { currentState.pressOffButton(this); }
  public void pressMuteButton() { currentState.pressMuteButton(this); }

  public void setState(State state) { currentState = state; }
}
```

The TV creates all state objects once and starts in OFF state.

4 OBJECTS (at Runtime)

Objects work together. TV delegates requests to the current state object.

TV (Context Object)

Holds current state and other state objects.

currentState → OffState
offState → OffState
onState → OnState
muteState → MuteState

OffState Object handles the request and may change the state.

OnState Object

MuteState Object

- Client (remote) calls a button on TV.
- TV forwards the request to the current state object.
- The state object does the work and may change the TV's current state.

RUNTIME FLOW (Example)



Start	Remote: press ON	Remote: press MUTE	Remote: press OFF	Remote: press OFF again
TV is created. Initial State = OFF (OffState is current)	`TV.pressOnButton()` OffState handles it. TV changes state to ON.	`TV.pressMuteButton()` OnState handles it. TV changes state to MUTE.	`TV.pressOffButton()` MuteState handles it. TV changes state to OFF.	`TV.pressOffButton()` OffState handles it. No state change (still OFF).
currentState → OffState	currentState → OnState	currentState → MuteState	currentState → OffState	currentState → OffState

Only the current state object handles the request. The TV (Context) never asks "which state am I?" – it simply delegates.

Exercise 02 — Online Order System (State Pattern)

SCENARIO

You are designing an **Online Order System**.

An order can be in one of the following states:

NEW | PAID | SHIPPED | DELIVERED | CANCELLED

The system supports these actions:

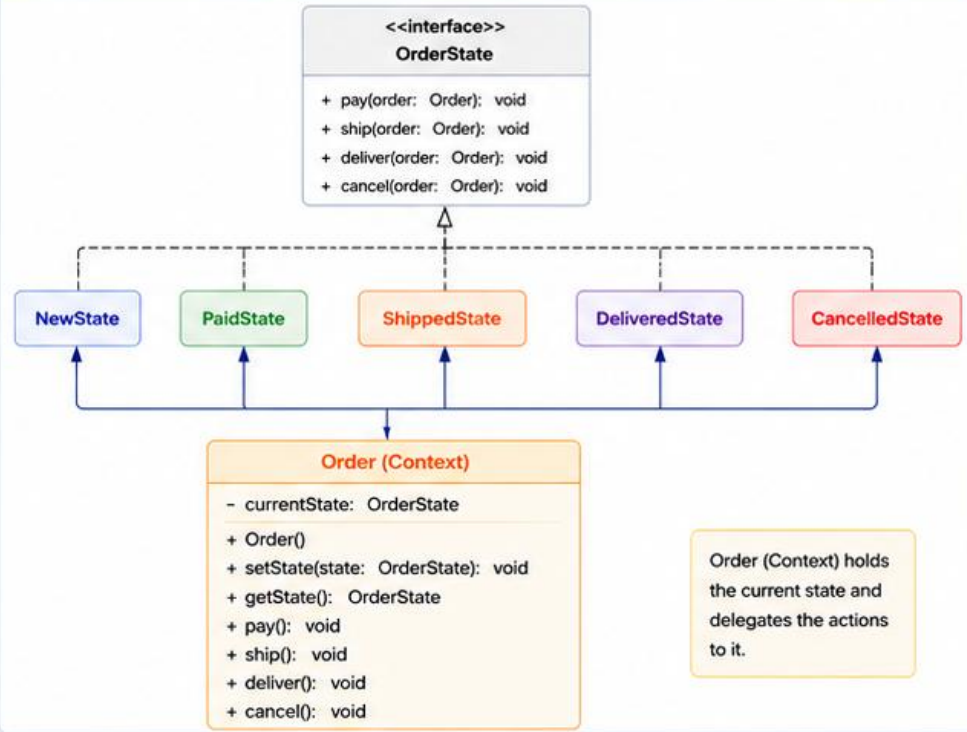
pay() | ship() | deliver() | cancel()

But the **behaviour** depends on the current state.

1) STEP 1 — BUILD THE STATE TABLE

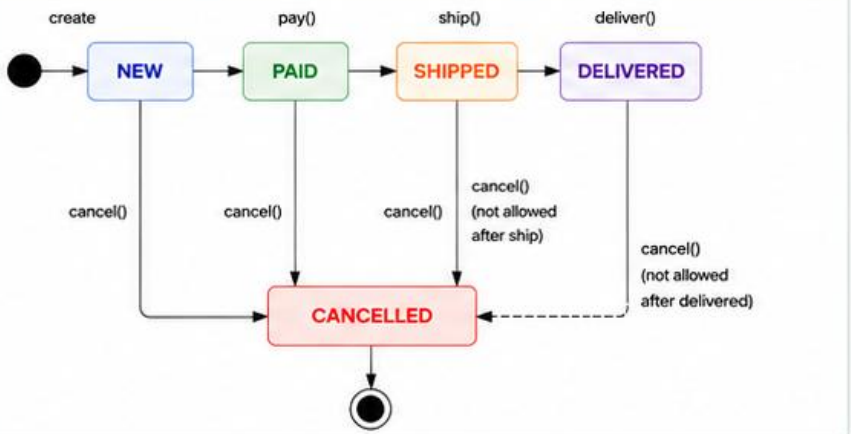
Current State	Action	Behaviour	Next State
NEW	pay()	Accept payment	PAID
	ship()	Cannot ship before payment	No change
	deliver()	Cannot deliver before ship	No change
	cancel()	Cancel the order	CANCELLED
PAID	pay()	Already paid	No change
	ship()	Ship the order	SHIPPED
	deliver()	Cannot deliver before ship	No change
	cancel()	Cancel and refund	CANCELLED
SHIPPED	pay()	Already paid	No change
	ship()	**Already shipped**	No change
	deliver()	**Deliver the order**	DELIVERED
	cancel()	Cannot cancel after ship	No change
DELIVERED	pay()	**Order already delivered**	No change
	ship()	**Order already delivered**	No change
	deliver()	**Order already delivered**	No change
	cancel()	**Order already delivered**	No change
CANCELLED	pay()	Order is cancelled	No change
	ship()	Order is cancelled	No change
	deliver()	Order is cancelled	No change
	cancel()	Order is cancelled	No change

2) STEP 2 — DRAW THE CLASS DESIGN



Order (Context) holds the current state and delegates the actions to it.

3) STEP 3 — STATE DIAGRAM (ORDER LIFECYCLE)



4) STEP 4 — TRACE THE BEHAVIOUR (EXAMPLE)

Scenario: Start with **NEW** order.

Execute: `pay() → ship() → deliver() → cancel()`

Step	Action	Current State (Before)	Handled By	Next State	Who Changes State?
1	pay()	NEW	NewState	PAID	NewState (order.setState)
2	ship()	PAID	PaidState	SHIPPED	**PaidState (order.setState)**
3	deliver()	SHIPPED	ShippedState	DELIVERED	**ShippedState (order.setState)**
4	cancel()	DELIVERED	DeliveredState	DELIVERED (No change)	No change

KEY IDEA

The Order (Context) never asks "Which state am I?"
It simply delegates the request to the current state object.

BENEFITS

- No complicated if-else or switch statements
- Easy to add new states
- Each state owns its own behaviour
- Follows OOP principles (OCP, SRP, Polymorphism)

Exercise 02 — Online Order System (State Pattern)

ORDER STATES

NEW | PAID | SHIPPED | DELIVERED | CANCELLED

ACTIONS SUPPORTED

pay() | ship() | deliver() | cancel()

But the **behaviour** depends on the current state.

BEHAVIOUR TRACE (EXAMPLE)

Start with **NEW** order. **Execute:** `pay() → ship() → deliver() → cancel()`

Step	Action	Current State (Before)	Next State	Who Changes State?
1	pay()	NEW	PAID	NewState (order.setState)
2	ship()	PAID	SHIPPED	PaidState (order.setState)
3	deliver()	SHIPPED	DELIVERED	ShippedState (order.setState)
4	cancel()	DELIVERED	DELIVERED (No change)	No change

JAVA IMPLEMENTATION (ALL CLASSES)

1) OrderState (Interface)

```
public interface OrderState {
    void pay(Order order);
    void ship(Order order);
    void deliver(Order order);
    void cancel(Order order);
}
```

2) Order (Context)

```
public class Order {
    private OrderState currentState;

    public Order() {
        this.currentState = new NewState(this);
    }

    public void setState(OrderState state) {
        this.currentState = state;
    }

    public OrderState getState() {
        return currentState;
    }

// delegate actions to current state
public void pay()     { currentState.pay(this); }
public void ship()    { currentState.ship(this); }
public void deliver() { currentState.deliver(this); }
public void cancel()  { currentState.cancel(this); }
}
```

3) NewState

```
public class NewState implements OrderState {
    public void pay(Order order) {
        System.out.println("Payment accepted.");
        order.setState(new PaidState());
    }

    public void ship(Order order) {
        System.out.println("Cannot ship before payment.");
    }

    public void deliver(Order order) {
        System.out.println("Cannot deliver before ship.");
    }

    public void cancel(Order order) {
        System.out.println("Order cancelled.");
        order.setState(new CancelledState());
    }
}
```

4) PaidState

```
public class PaidState implements OrderState {
    public void pay(Order order) {
        System.out.println("Already paid.");
    }

    public void ship(Order order) {
        System.out.println("Order shipped.");
        order.setState(new ShippedState());
    }

    public void deliver(Order order) {
        System.out.println("Cannot deliver before ship.");
    }

    public void cancel(Order order) {
        System.out.println("Order cancelled.\n
        "Refund issued.");
        order.setState(new CancelledState());
    }
```

5) ShippedState

```
public class ShippedState implements OrderState {
    public void pay(Order order) {
        System.out.println("Already paid.");
    }

    public void ship(Order order) {
        System.out.println("Already shipped.");
    }

    public void deliver(Order order) {
        System.out.println("Order delivered.");
        order.setState(new DeliveredState());
    }

    public void cancel(Order order) {
        System.out.println("Cannot cancel after
        "shipping.");
    }
}
```

6) DeliveredState

```
public class DeliveredState implements OrderState {
    public void pay(Order order) {
        System.out.println("Order already delivered.");
    }

    public void ship(Order order) {
        System.out.println("Order already delivered.");
    }

    public void deliver(Order order) {
        System.out.println("Order already delivered.");
    }

    public void cancel(Order order) {
        System.out.println("Cannot cancel after
                           "delivered.");
    }
}
```

7) CancelledState

```
public class CancelledState implements OrderState {
    public void pay(Order order)     { System.out.println("Order is cancelled."); }
    public void ship(Order order)    { System.out.println("Order is cancelled."); }
    public void deliver(Order order) { System.out.println("Order is cancelled."); }
    public void cancel(Order order)  { System.out.println("Order is cancelled."); }
}
```

8) Client (Main Class)

```
public class Client {
    public static void main(String[] args) {
        Order order = new Order();

        order.pay();            // NEW    -> PAID
        order.ship();           // PAID   -> SHIPPED
        order.deliver();        // SHIPPED -> DELIVERED
        order.cancel();         // DELIVERED (no change)

        System.out.println("Final State: " + order.getState().getClass().getSimpleName());
    }
}
```

What Changes When We Add a State?

ADD: RETURNED

WITHOUT STATE PATTERN

Order

- pay()
- ship()
- deliver()
- cancel()
- return() ← NEW

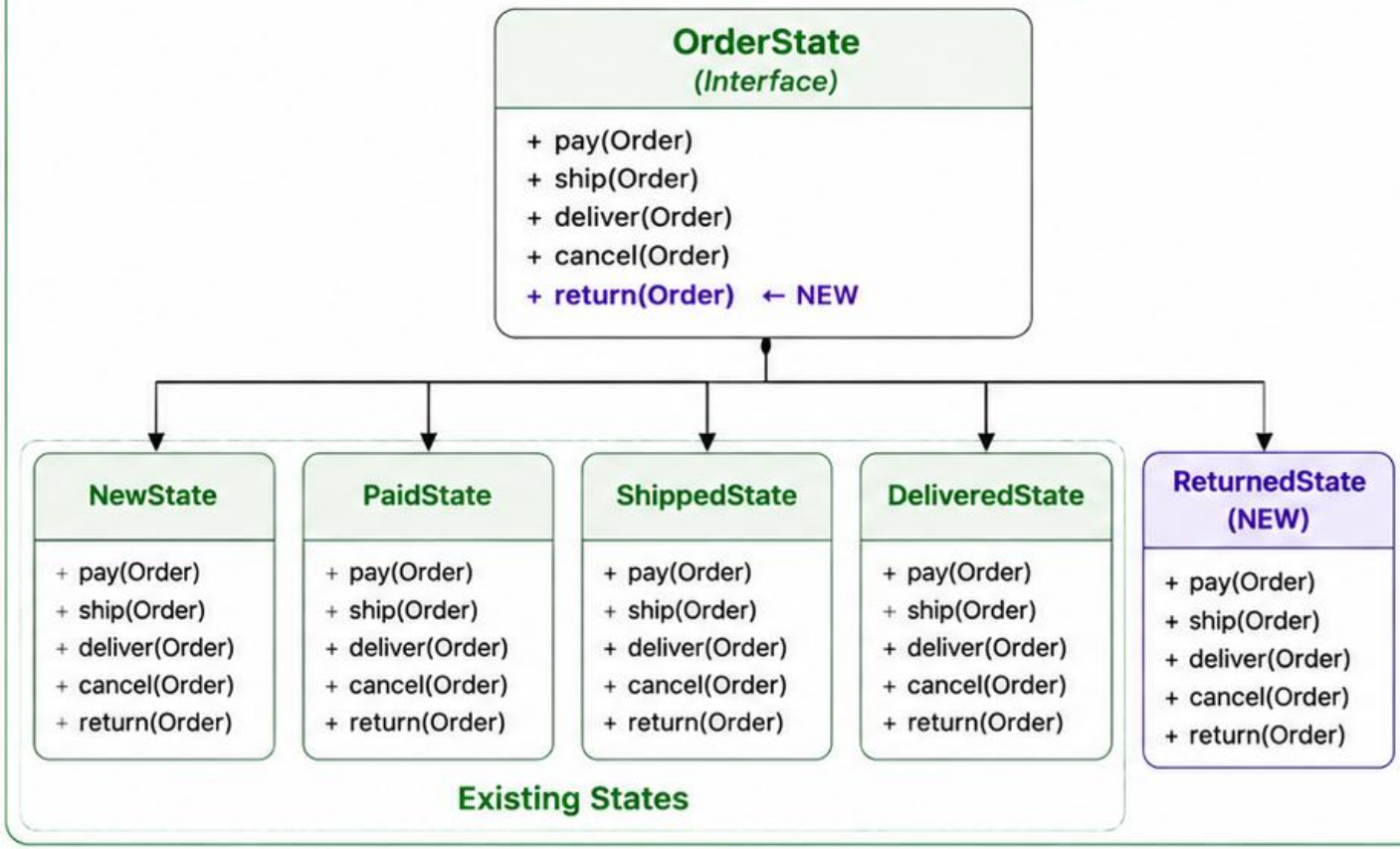
↓

More conditions

Adding **RETURNED** means modifying Order class:

- More if/else or switch cases
- Higher risk of bugs
- Harder to maintain and test

WITH STATE PATTERN



DISCUSSION

Which existing classes should change? → Ideally: **Very few or none.**

This extensibility is one of the major advantages identified in the previous lecture.

When NOT to Use It - Don't Turn Every `if` Into a State

STATE PATTERN IS USEFUL WHEN:

- Behaviour changes **significantly** by state
- States are **well-defined**
- **Transitions** matter
- State-specific logic is becoming **complex**
- **New states** are likely to be added

AVOID IT WHEN:

- 2–3 There are only **2–3 simple** states
- = Behaviour **barely** changes
- States **rarely** change
- </> A few simple conditions are **clearer**

Your previous lecture makes the same distinction:

Simple, stable states may not justify the additional classes,
while **complex state-dependent behaviour** is a stronger fit.

The Rule to Remember

DON'T ASK:

> "Can I use the State Pattern here?"

ASK:

> "Is this object's behaviour fundamentally different depending on its current state?"

IF YES:

- State
- State-specific behaviour
- Polymorphism
- Cleaner Context
- Easier extension

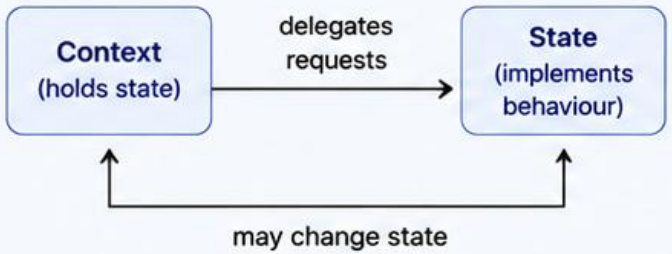
FINAL TAKEAWAY

> Move behaviour that changes with state into objects representing those states.

State Pattern – Summary

1 WHAT IS IT?

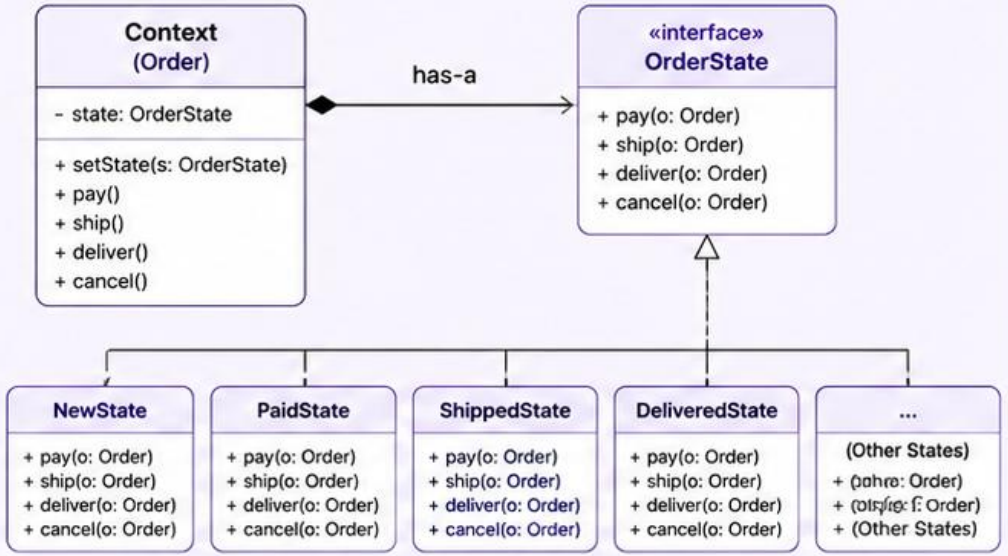
The State Pattern allows an object to **alter its behaviour** when its internal state changes.



2 WHY USE IT?

- Encapsulates **state-specific behaviour** in separate classes
- **Eliminates** large conditional statements
- Makes code easier to **maintain** and **extend**
- Supports **adding new states** with minimal changes
- Each state has **clear responsibility** and behaviour

3 KEY STRUCTURE



4 WHEN TO USE IT?

- Behaviour changes **significantly** by state
- States are **well-defined**
- Transitions between states **matter**
- State-specific logic is becoming **complex**
- New states are **likely** to be added

5 WHEN NOT TO USE IT?

- Only **2–3 simple** states
- Behaviour **barely** changes
- States **rarely** change
- A few simple conditions are **clearer**
- Too many states can be **hard to manage**

6 THE RULE TO REMEMBER

> Is this object's behaviour **fundamentally different** depending on its **current state**?

If yes → **Use State Pattern**

KEY TAKEAWAY

> Move behaviour that changes with state into objects representing those states.
> This leads to **cleaner code**, **easier extension**, and better **separation of concerns**.